

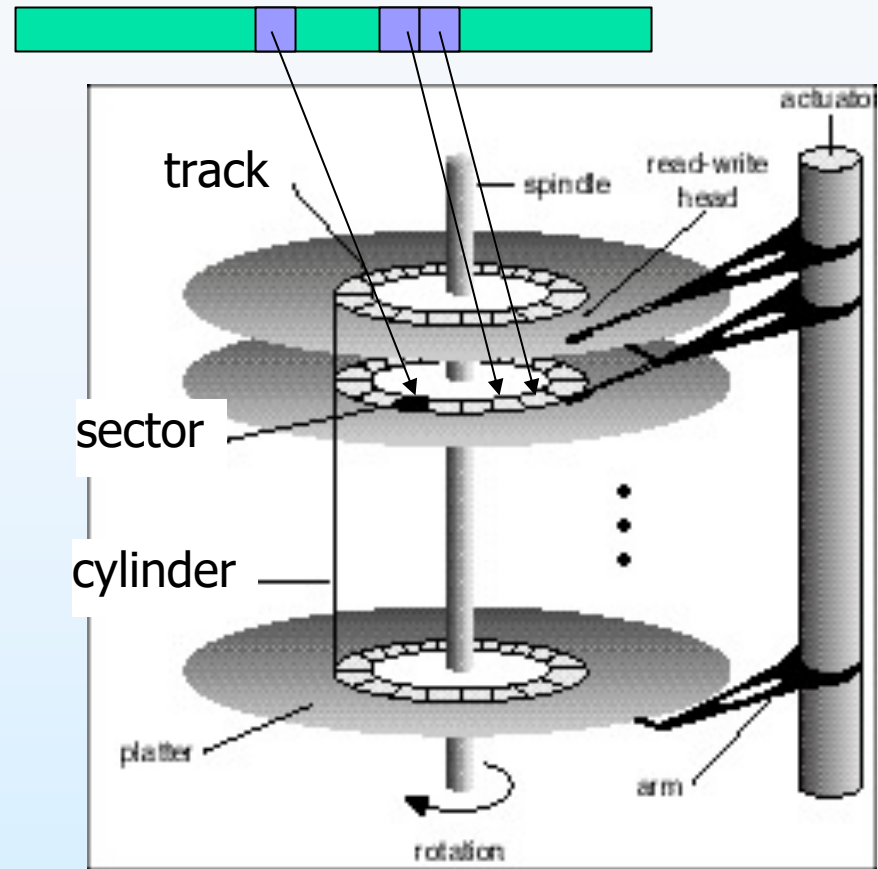
Chapter 11: File System Implementation





Chapter 11: File System Implementation

- File-System Structure
- File-System Implementation
- Directory Implementation
- Allocation Methods
- Free-Space Management
- Efficiency and Performance
- Recovery
- Log-Structured File Systems
- NFS
- Example: WAFL File System





Objectives

- ❑ To describe the details of implementing local file systems and directory structures
- ❑ To describe the implementation of remote file systems
- ❑ To discuss block allocation and free-block algorithms and trade-offs





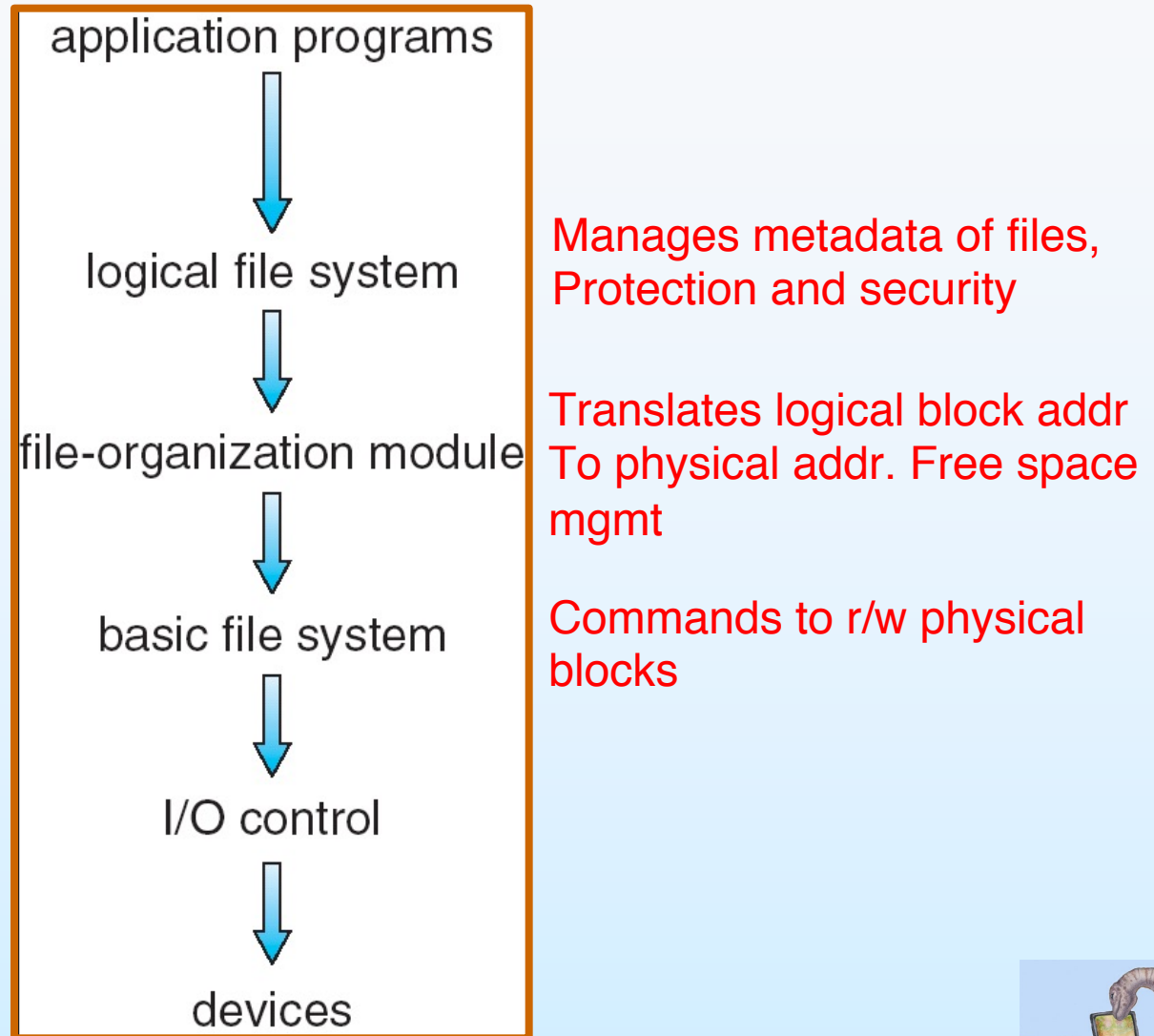
File-System Structure

- File structure
 - Logical storage unit
 - Collection of related information
- File system resides on secondary storage (disks)
- File system organized into layers
- **File control block** – storage structure consisting of information about a file





Layered File System





A Typical File Control Block

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks





Data Structures Used to Implement FS

- Disk structures
 - Boot control block
 - Volume control block
 - Directory structure per file system
 - Per-file FCB (**inode** in UFS, **master file table** entry in NTFS)
- In-memory structures
 - In-memory mount table about each mounted volume
 - Directory cache
 - System-wide open-file table
 - Per-process open-file table





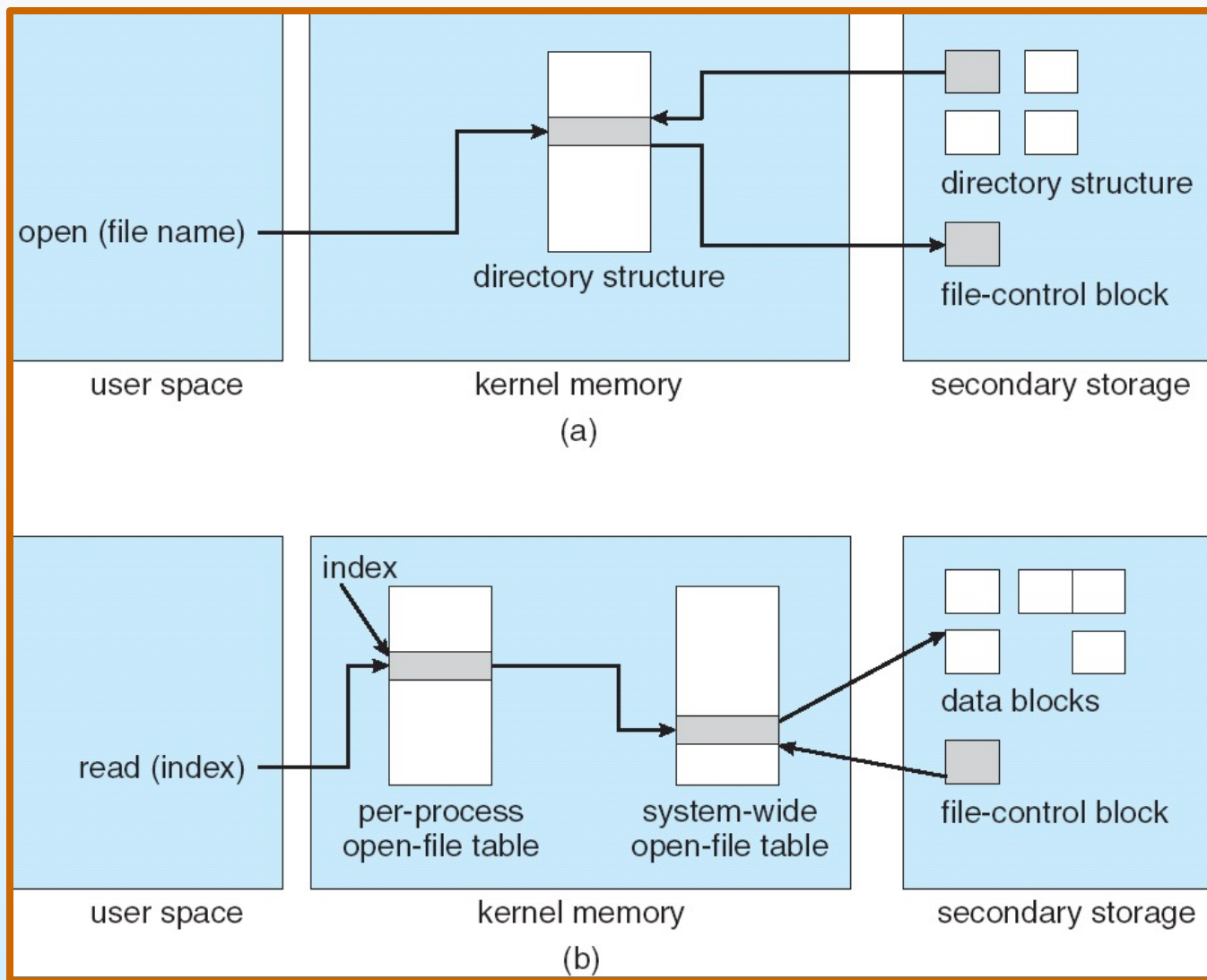
In-Memory File System Structures

- The following figure illustrates the necessary file system structures provided by the operating systems.
- Figure 12-3(a) refers to opening a file.
- Figure 12-3(b) refers to reading a file.





In-Memory File System Structures





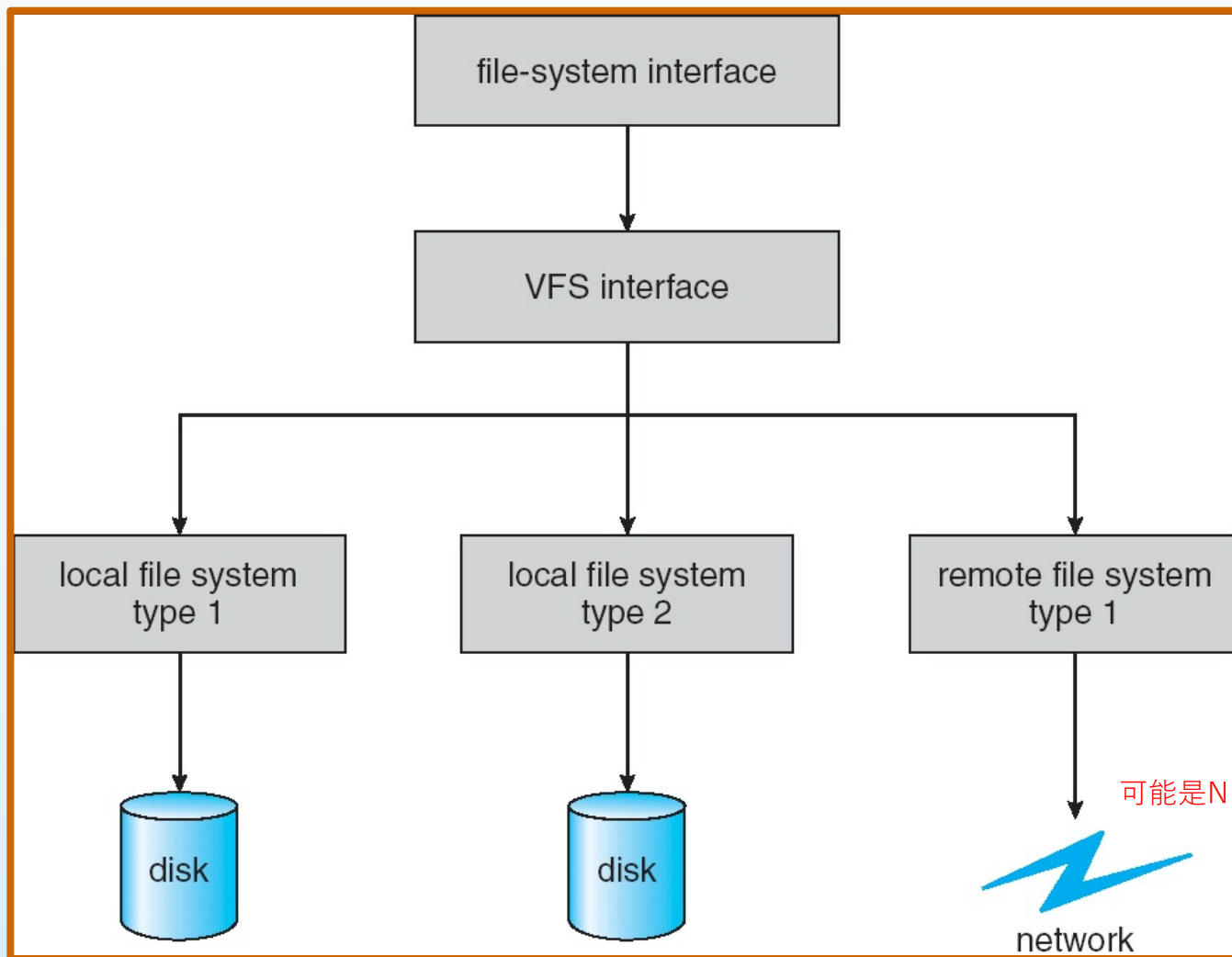
Virtual File Systems

- ❑ Virtual File Systems (VFS) provide an object-oriented way of implementing file systems.
- ❑ VFS allows the same system call interface (the API) to be used for different types of file systems.
- ❑ The API is to the VFS interface, rather than any specific type of file system.
- ❑ Defines a network-wide unique structure called **vnode**.





Schematic View of Virtual File System



可能是NDFS





Directory Implementation

- **Linear list** of file names with pointer to the data blocks.
 - simple to program
 - time-consuming to execute
- **Hash Table** – linear list with hash data structure.
 - decreases directory search time
 - **collisions** – situations where two file names hash to the same location
 - fixed size – can use chained-overflow hash table

tradeoff





Allocation Methods

- An allocation method refers to how disk blocks are allocated for files:
- **Contiguous allocation**
- **Linked allocation**
- **Indexed allocation**





Contiguous Allocation

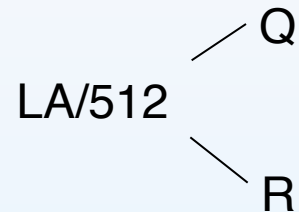
- ❑ Each file occupies a set of contiguous blocks on the disk
- ❑ Simple – only starting location (block #) and length (number of blocks) are required
- ❑ Random access supported
- ❑ Wasteful of space (dynamic storage-allocation problem)
- ❑ Files cannot grow





Contiguous Allocation

- Mapping from logical to physical



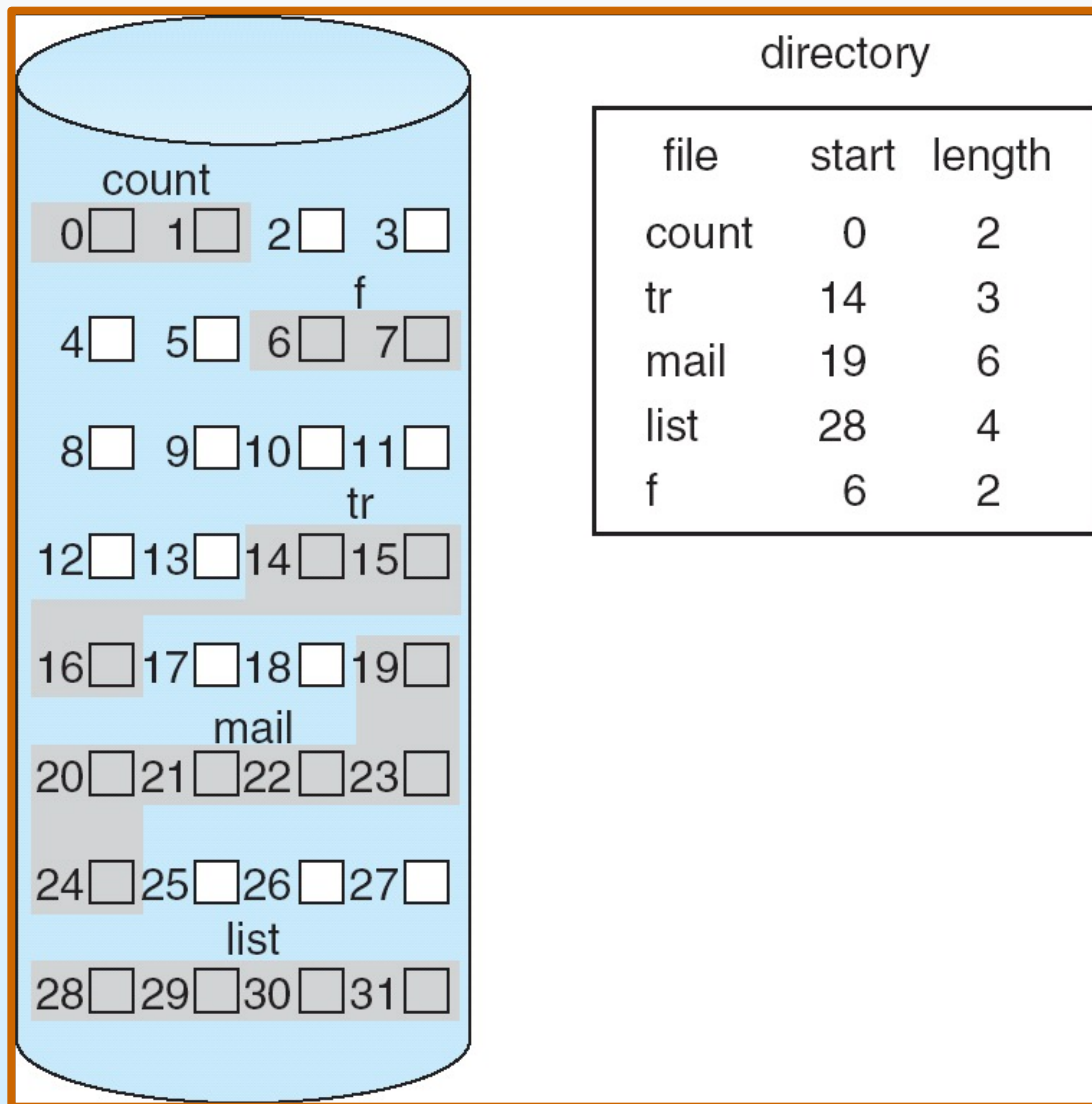
Block to be accessed = $Q + \text{start_address}$

Displacement into block = R





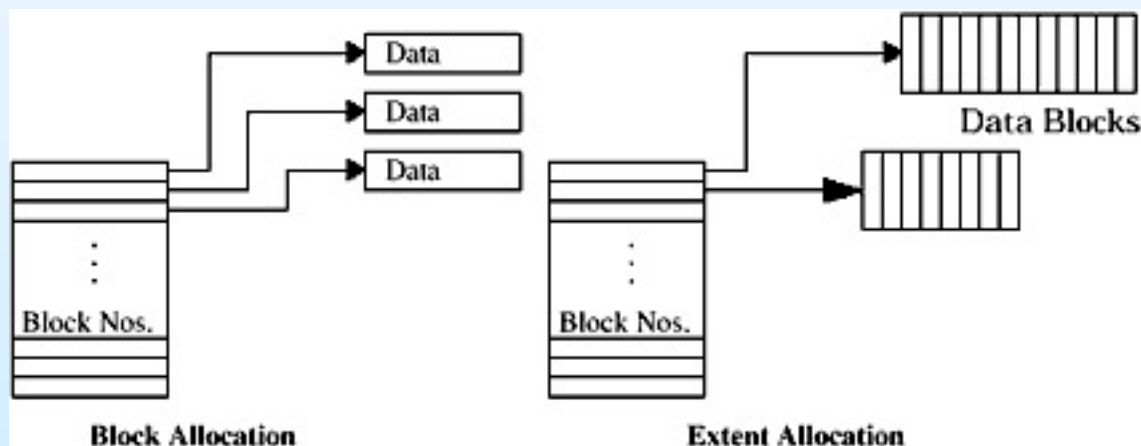
Contiguous Allocation of Disk Space





Extent-Based Systems

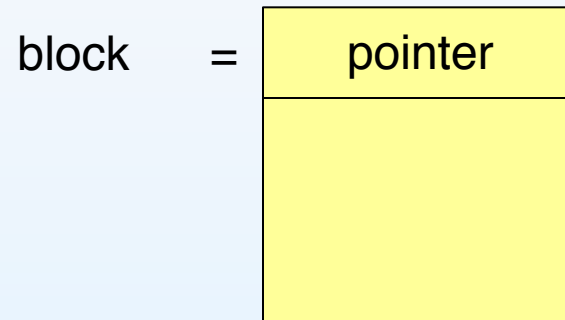
- ❑ Many newer file systems (I.e. Veritas File System) use a modified contiguous allocation scheme
- ❑ Extent-based file systems allocate disk blocks in **extents**
- ❑ An **extent** is a contiguous block of disks
 - ❑ Extents are allocated for file allocation
 - ❑ A file consists of one or more extents.





Linked Allocation

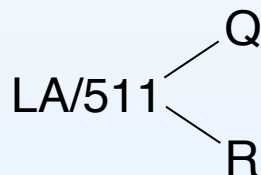
- Each file is a linked list of disk blocks: blocks may be scattered anywhere on the disk.





Linked Allocation (Cont.)

- Simple – need only starting address
- Free-space management system – no waste of space
- No random access, poor reliability
- Mapping



Block to be accessed is the Qth block in the linked chain of blocks representing the file.

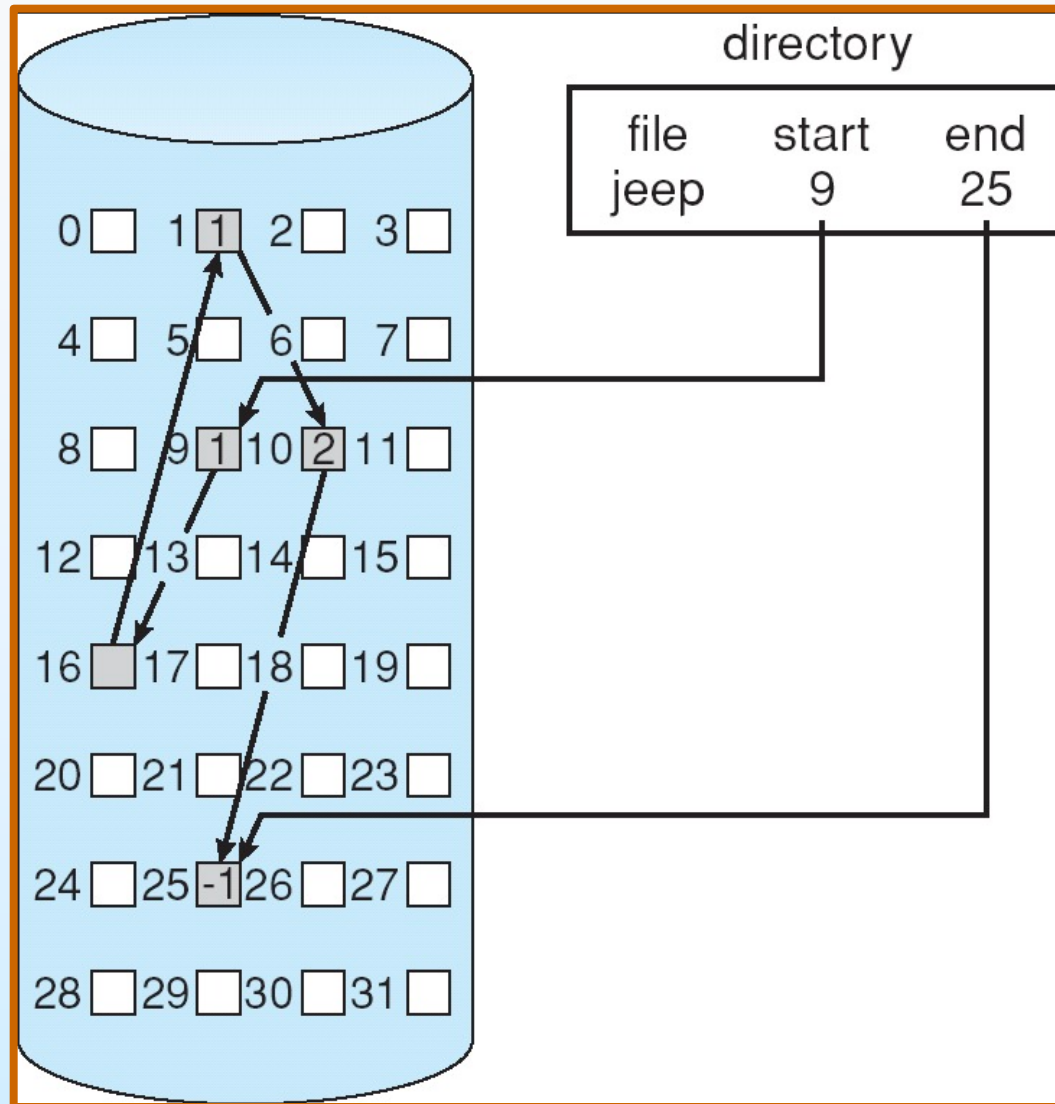
Displacement into block = $R + 1$

File-allocation table (FAT) – disk-space allocation used by MS-DOS and OS/2.



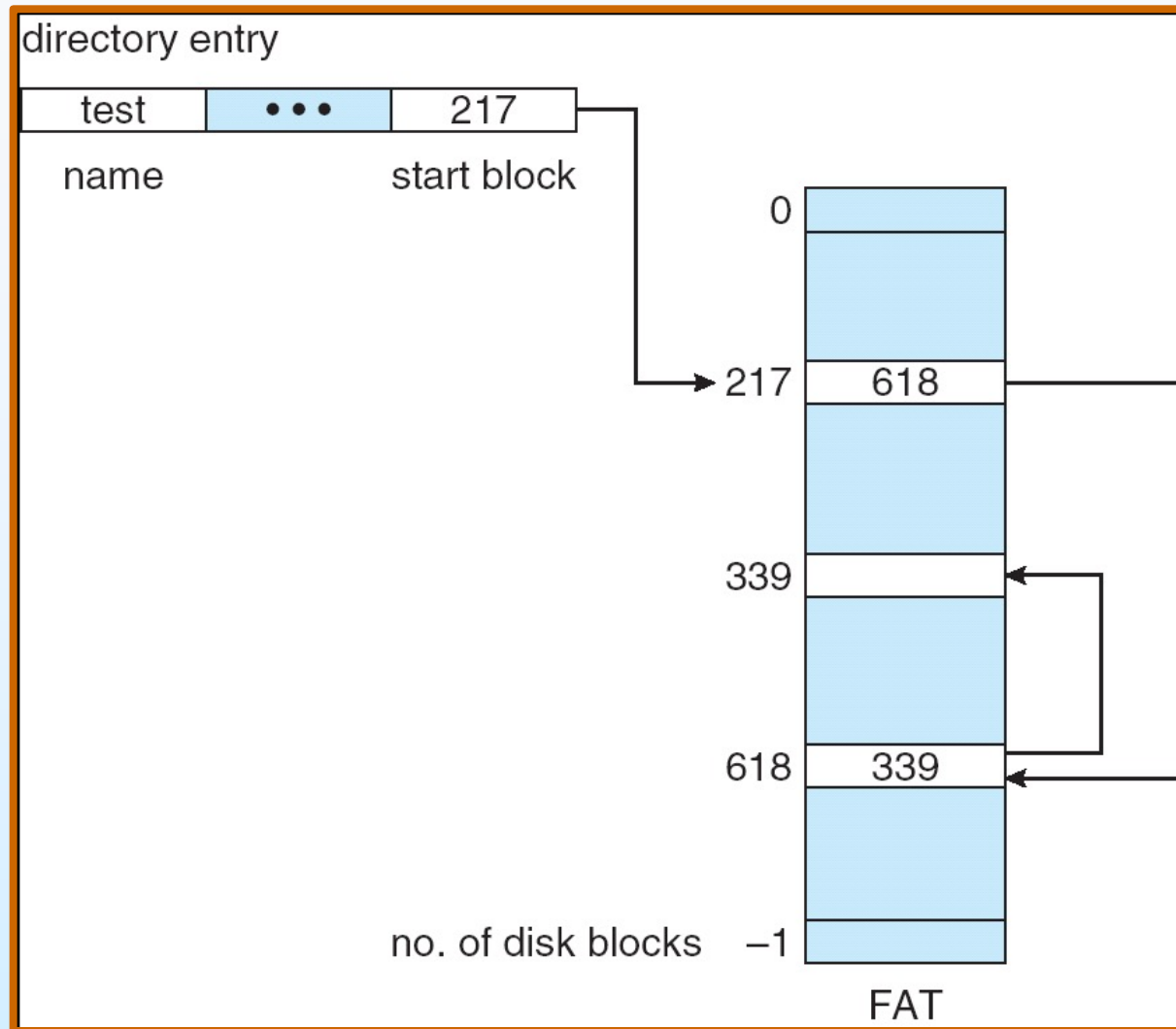


Linked Allocation





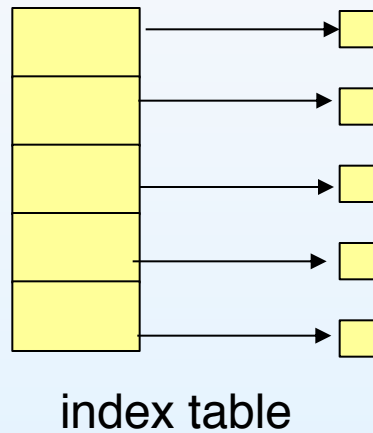
File-Allocation Table





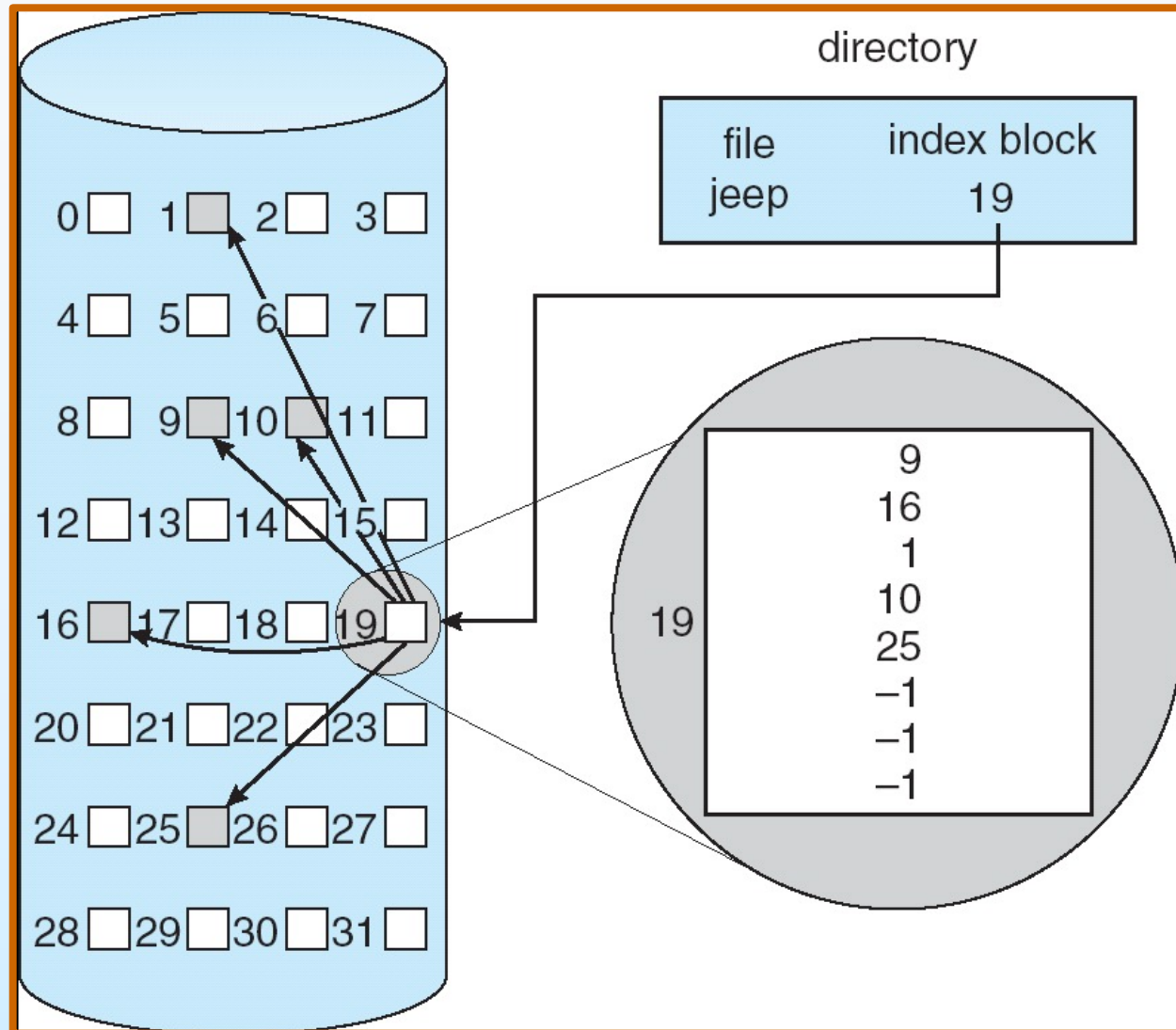
Indexed Allocation

- Brings all pointers together into the *index block*.
- Logical view.



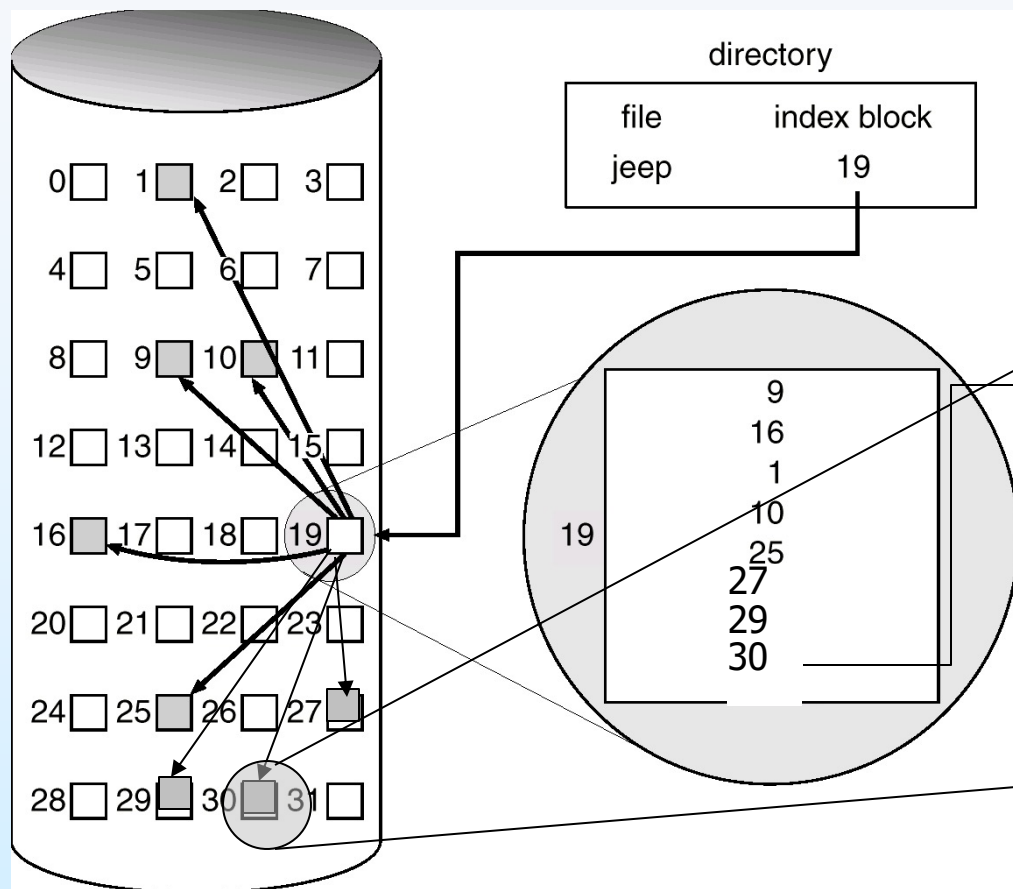


Example of Indexed Allocation





Example of Indexed Allocation (Cont.)

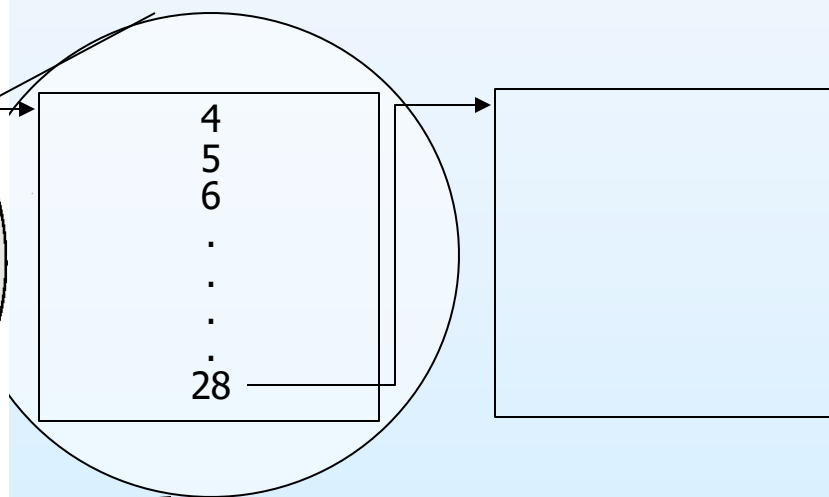


Advantage:

- Adjustable to any size of files

Disadvantages:

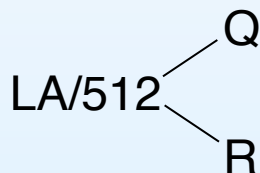
- Slower random accesses for larger files





Indexed Allocation (Cont.)

- ❑ Need index table (analogous to **page table**)
- ❑ Random access
- ❑ Dynamic access without external fragmentation, but have overhead of index block.
- ❑ When mapping from logical to physical in a file of maximum size of 256K words and block size of 512 words ($512=2^9$, $2^9 * 2^9 = 2^{18}$, $2^{18} / 1024 = 256$). We need only 1 block for index table.



Q = displacement into index table

R = displacement into block





Indexed Allocation – Mapping (Cont.)

- When mapping from logical to physical in a file of unbounded length (block size of 512 words). – more pointers are needed
- Linked scheme – Link blocks of index table (no limit on size).

$$LA / (512 \times 511) \begin{cases} Q_1 \\ R_1 \end{cases}$$

Q_1 = block of index table

R_1 is used as follows:

$$R_1 / 512 \begin{cases} Q_2 \\ R_2 \end{cases}$$

Q_2 = displacement into block of index table

R_2 displacement into block of file:





Indexed Allocation – Mapping (Cont.)

- Two-level index (maximum file size is 512^3)

$$LA / (512 \times 512) \begin{cases} Q_1 \\ R_1 \end{cases}$$

Q_1 = displacement into outer-index

R_1 is used as follows:

$$R_1 / 512 \begin{cases} Q_2 \\ R_2 \end{cases}$$

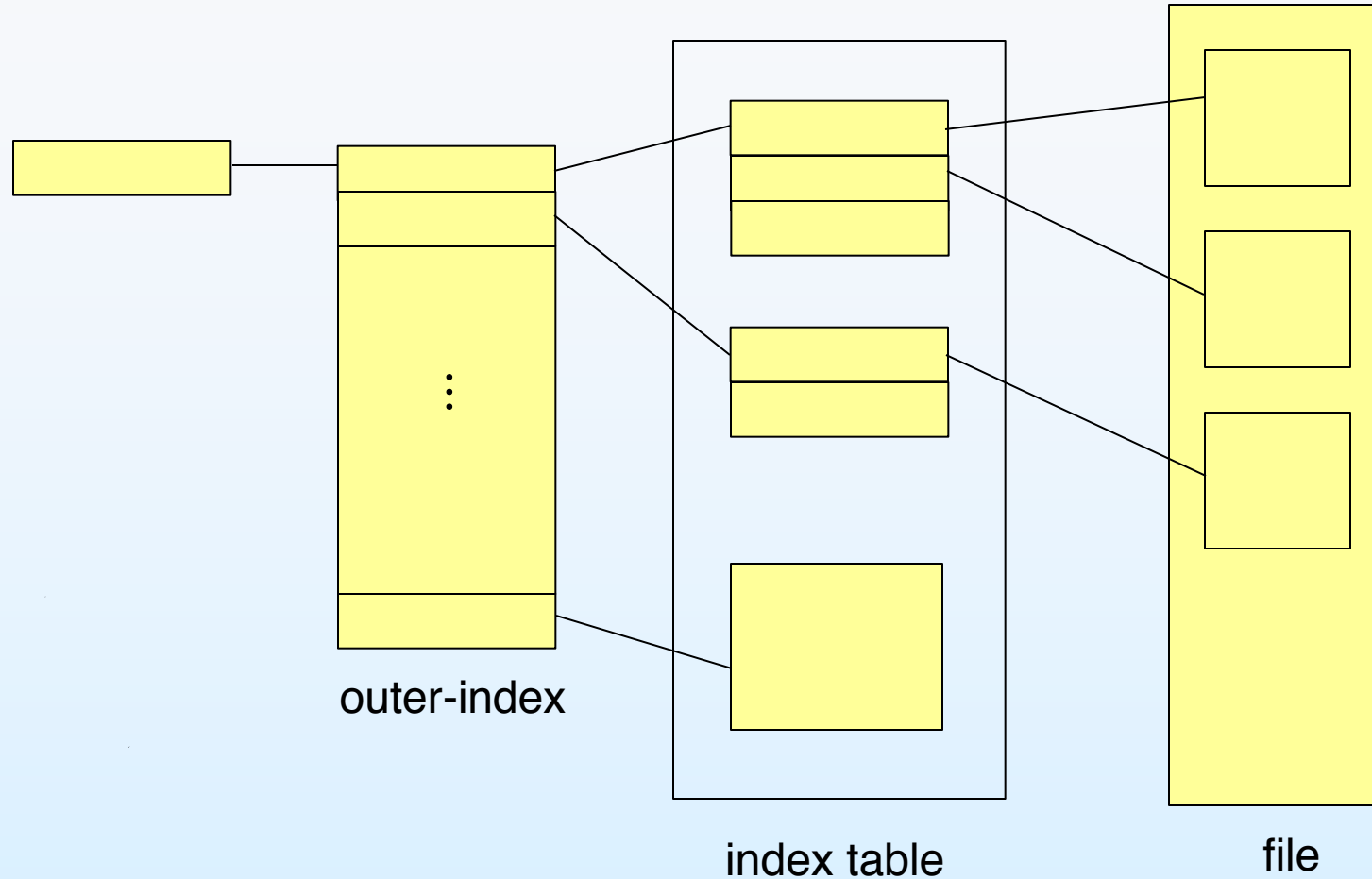
Q_2 = displacement into block of index table

R_2 displacement into block of file:



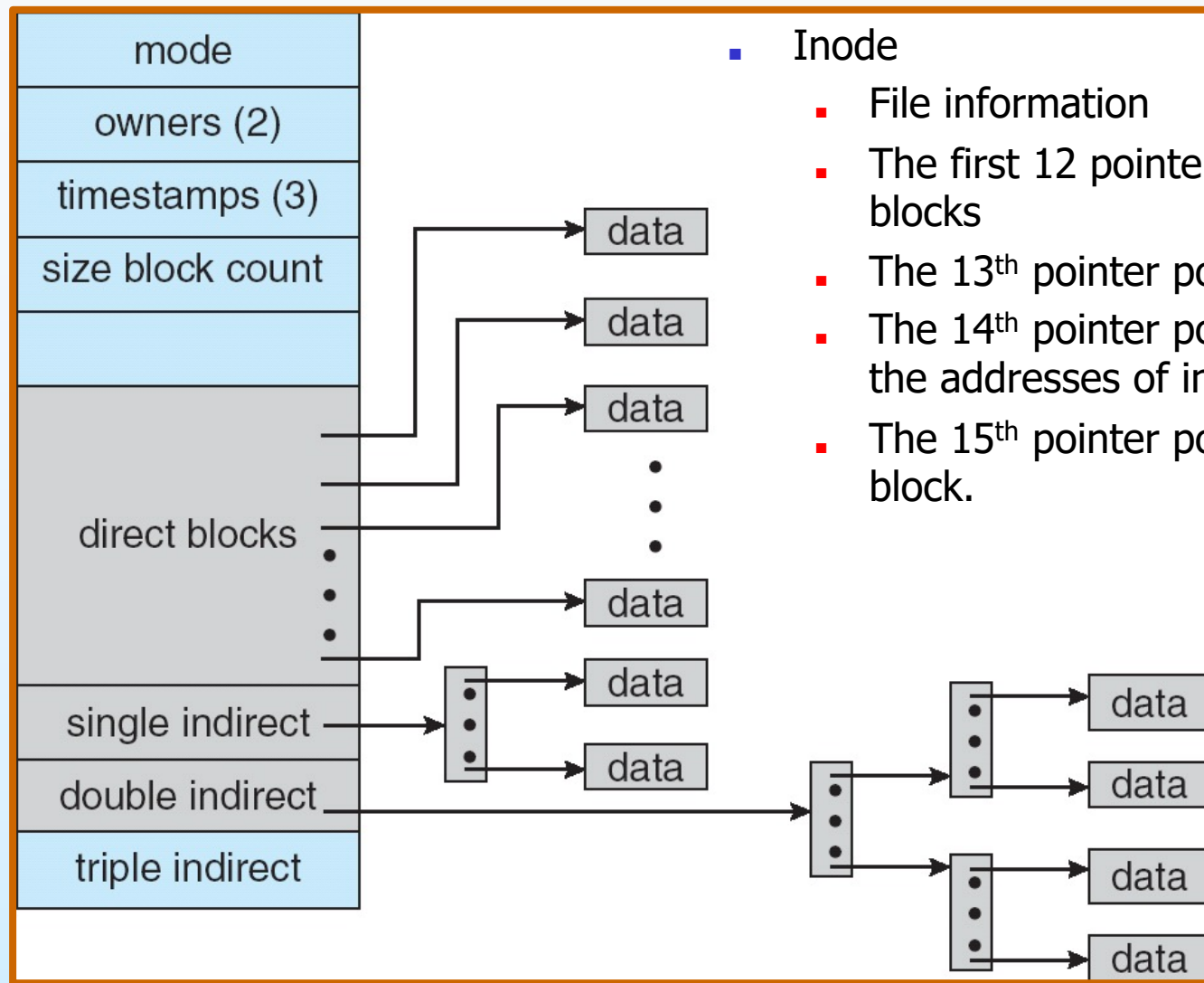


Indexed Allocation – Mapping (Cont.)





Combined Scheme: UNIX (4K bytes per block)





File System

- » Consider a UNIX filesystem with the following components:
 - ❑ Disk blocks are 4096 bytes. Sectors are 512 bytes long.
 - ❑ All metadata pointers are 32-bits long.
 - ❑ An inode has 12 direct block pointers, one indirect block pointer and one double-indirect block pointer.
 - ❑ The total inode size is 256 bytes.
 - ❑ Both indirect and double indirect blocks take up an entire disk block.





File System

- » How much disk space, including metadata and data blocks, is needed to store a 4 GB DVD image file? You can leave the answer in symbolic (e.g., 6MB + 3KB) form

Requires 2^{32} bytes of actual data: 1048576 (2^{20}) data blocks.

1024 block ptrs per indirect block.

$1,048,564 \text{ blocks} / 1024 = 1024 \text{ indirect blocks needed.}$

1 double-indirect block needed.

256 bytes for inode.

Total: 4,299,165,952 bytes =

*$256b + 1 * 4096b + 1024 * 4096b + 1024 * 1024 * 4096b$*





- | | | | | | | | | | | |
|---|---|---|--|--|--|--|--|-----|--|-----|
| 0 | 1 | 2 | | | | | | ... | | n-1 |
|---|---|---|--|--|--|--|--|-----|--|-----|

$$\text{bit}[i] = \begin{cases} 1 \Rightarrow \text{block}[i] \text{ free} \\ 0 \Rightarrow \text{block}[i] \text{ occupied} \end{cases}$$

(number of bits per word) *
(number of 0-value words) +
offset of first 1 bit





Free-Space Management (Cont.)

- Bit map requires extra space

- Example:

block size = 2^{12} bytes

disk size = 2^{30} bytes (1 gigabyte)

$n = 2^{30}/2^{12} = 2^{18}$ bits (or 32K bytes)

- Easy to get contiguous files

- Linked list (free list) – see figure

- Cannot get contiguous space easily
 - But basically can work (FAT)
 - No waste of space

- Grouping – a modification of the Linked List

- Addresses of the n free blocks are stored in the first block.
 - The first $n-1$ blocks are actually free. The last block contains addresses of another n free blocks

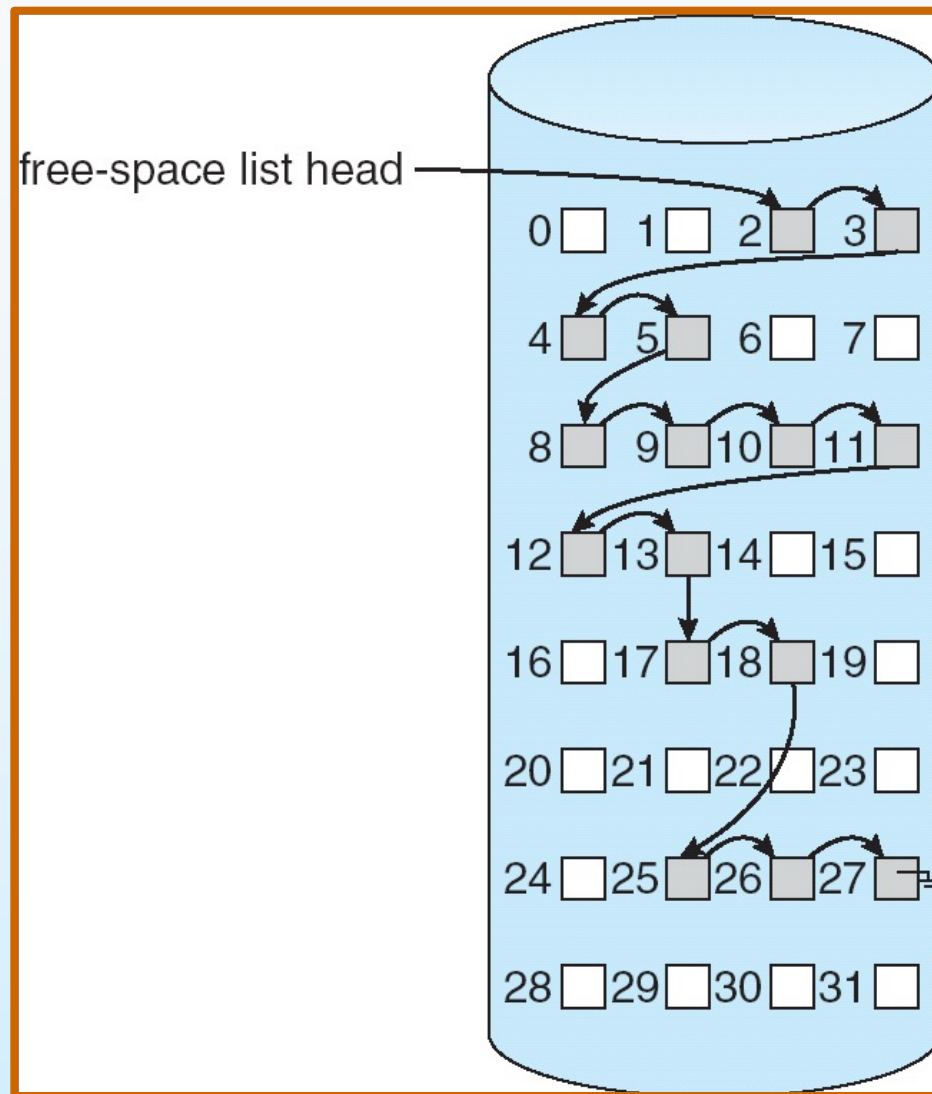
- Counting

- Address of the first free block and number n contiguous blocks





Linked Free Space List on Disk

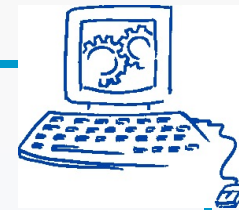




Free-Space Management (Cont.)

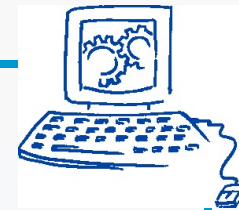
- Need to protect:
 - Pointer to free list
 - Bit map
 - ▶ Must be kept on disk
 - ▶ The copy in memory and disk may differ
 - ▶ Cannot allow for block[*i*] to have a situation where bit[*i*] = 1 in memory and bit[*i*] = 0 on disk
- Solution:
 - ▶ Set bit[*i*] = 1 in disk
 - ▶ deallocate block[*i*]
 - ▶ Set bit[*i*] = 1 in memory





File System Problem 1

- Consider a file system on a disk that has both logical and physical block sizes of 512 words. Assume that the information about each file is already in memory. For each of the three allocation strategies (contiguous, linked, and indexed), answer these questions:
 - a. How is the logical-to-physical address mapping accomplished in this system? (For the indexed allocation, assume that a file is always less than 512 blocks long.)
 - b. If we are currently at logical block 10 (the last block accessed was block 10) and want to access logical block 4, how many physical blocks must be read from the disk?



File System Problem 1

- Let z to be the starting file address.

- contiguous:

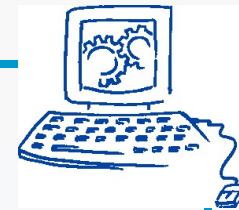
(a)

Divide the logical address by 512 with x and y , y is the remainder.

$x+z$ is the block number, y is the offset

(b)

Directly access the block 4. 1 block blocks must be read.



File System Problem 1

● linked:

(a)

Divide the logical physical address by 511 with x and y , y is the remainder.

Search the blocks down $x+1$, and the $y+1$ is the displacement into the last physical block.

(b)

Search for the list. 4 blocks must be read.



File System Problem 1

- indexed:

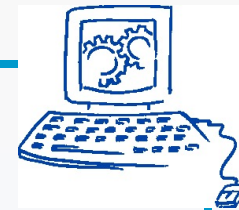
(a)

Divide the logical address by 512 with x and y , y is the remainder.

Physical block address is contained in the index block at location x . y is the offset of the block.

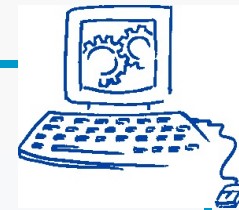
(b)

2 blocks must be read. One for index block and one for the block needed.



File System Problem 2

- Some file systems allow disk storage to be allocated at different levels of granularity. For instance, a file system could allocate 4 KB of disk space as a single 4-KB block or as eight 512-byte blocks. How could we take advantage of this flexibility to improve performance? What modifications would have to be made to the free-space management scheme in order to support this feature?



File System Problem 2

- This schema decrease internal fragmentation.
- In addition to maintaining a **bitmap** of free blocks, extra state of which the sub blocks are currently being used is necessary. The allocator would have to check all the state of sub blocks if a larger block is needed and coalesce the sub blocks when sub blocks becomes free.



Efficiency and Performance

- Efficiency dependent on:
 - **disk allocation** and **directory** algorithms
 - types of data kept in file's directory entry (for example "last write date" is recorded in directory)
- **Generally, every data item has to be considered for its effect.**
- Performance
 - **disk cache** – separate section of main memory for frequently used blocks
 - **free-behind and read-ahead** – techniques to optimize sequential access
 - improve PC performance by dedicating section of memory as virtual disk, or **RAM disk**





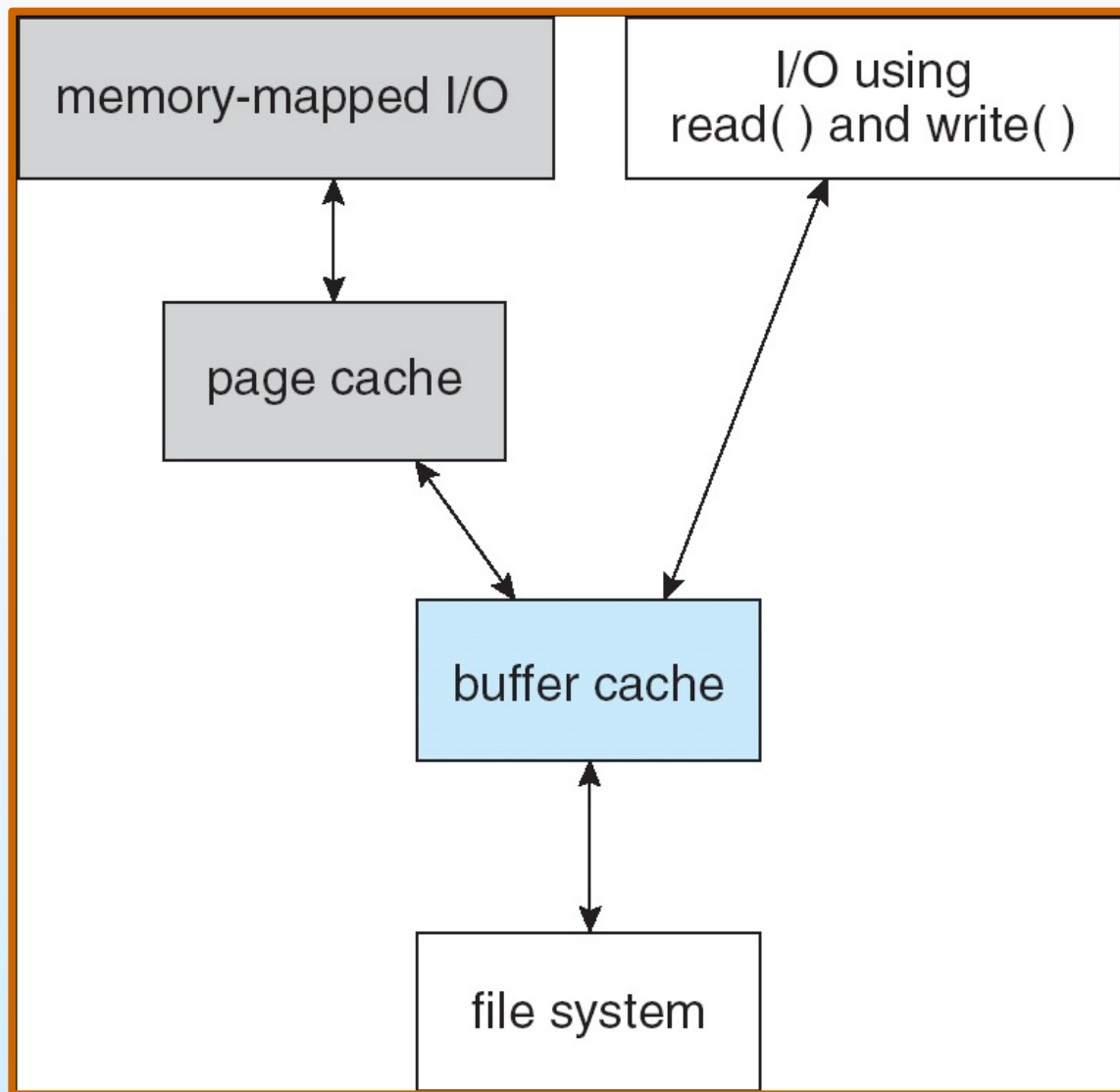
Page Cache

- A **page cache** caches pages rather than disk blocks using **virtual memory** techniques
- Memory-mapped I/O uses a page cache
- Routine I/O through the file system uses the buffer (disk) cache
- This leads to the following figure





I/O Without a Unified Buffer Cache





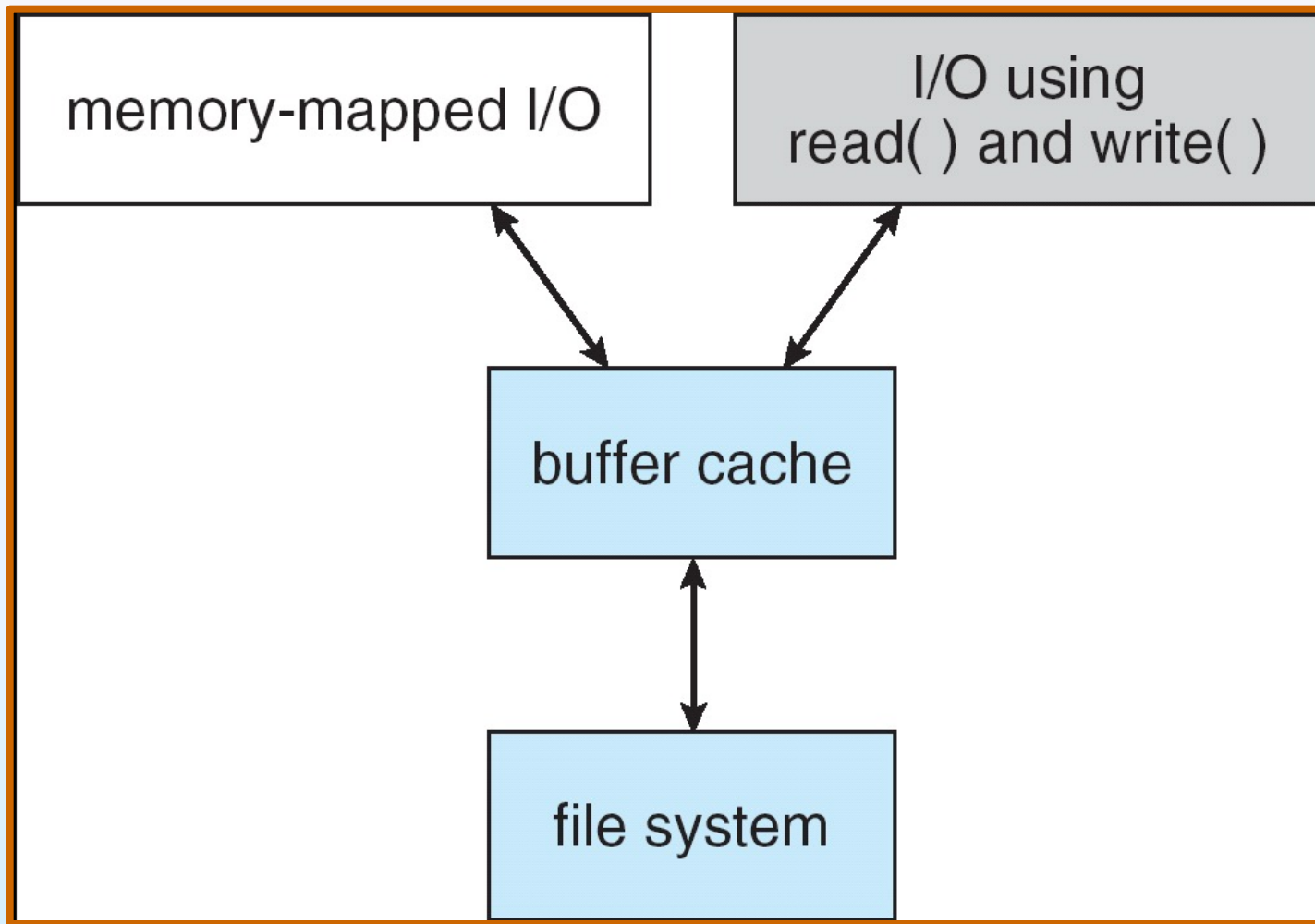
Unified Buffer Cache

- A unified buffer cache uses the same page cache to cache both **memory-mapped pages** and ordinary **file system I/O**
- Avoids double caching





I/O Using a Unified Buffer Cache





Recovery

- Consistency checking – compares data in directory structure with data blocks on disk, and tries to fix inconsistencies
- Use system programs to **back up** data from disk to another storage device (floppy disk, magnetic tape, other magnetic disk, optical)
- Recover lost file or disk by **restoring** data from backup. **Full backup** and N **incremental backups** for convenience.





Log Structured File Systems

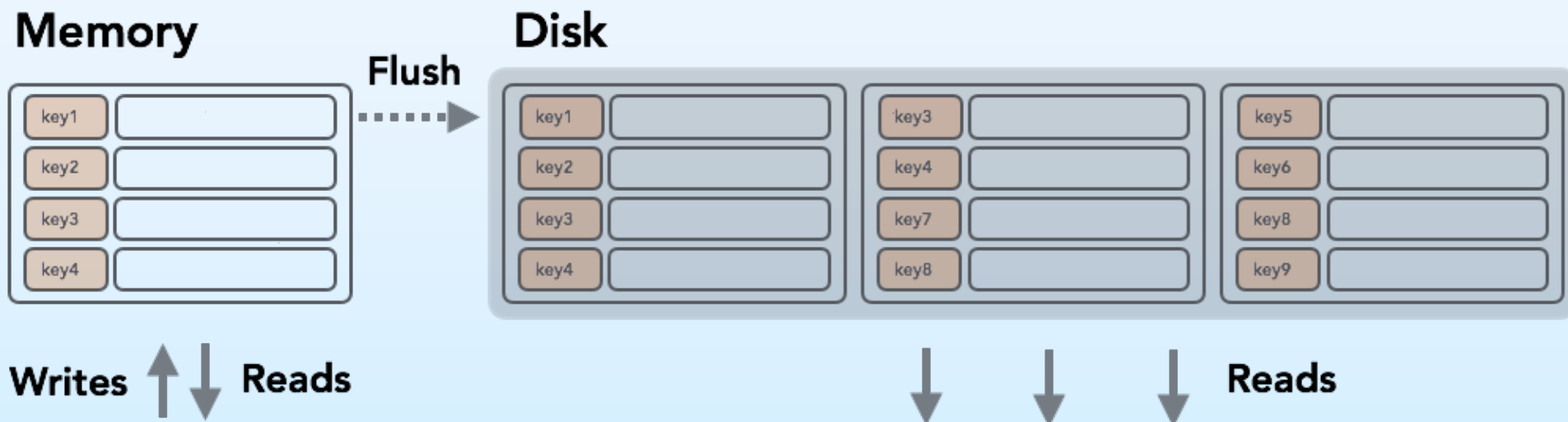
- **Log structured** (or journaling) file systems record each update to the file system as a **transaction**
- All transactions are written to a **log**
 - A transaction is considered **committed** once it is written to the log
 - However, the file system may not yet be updated
- The transactions in the log are asynchronously written to the file system
 - When the file system is modified, the transaction is removed from the log
- If the file system crashes, all remaining transactions in the log must still be performed





Log Structured Merge (LSM) Tree

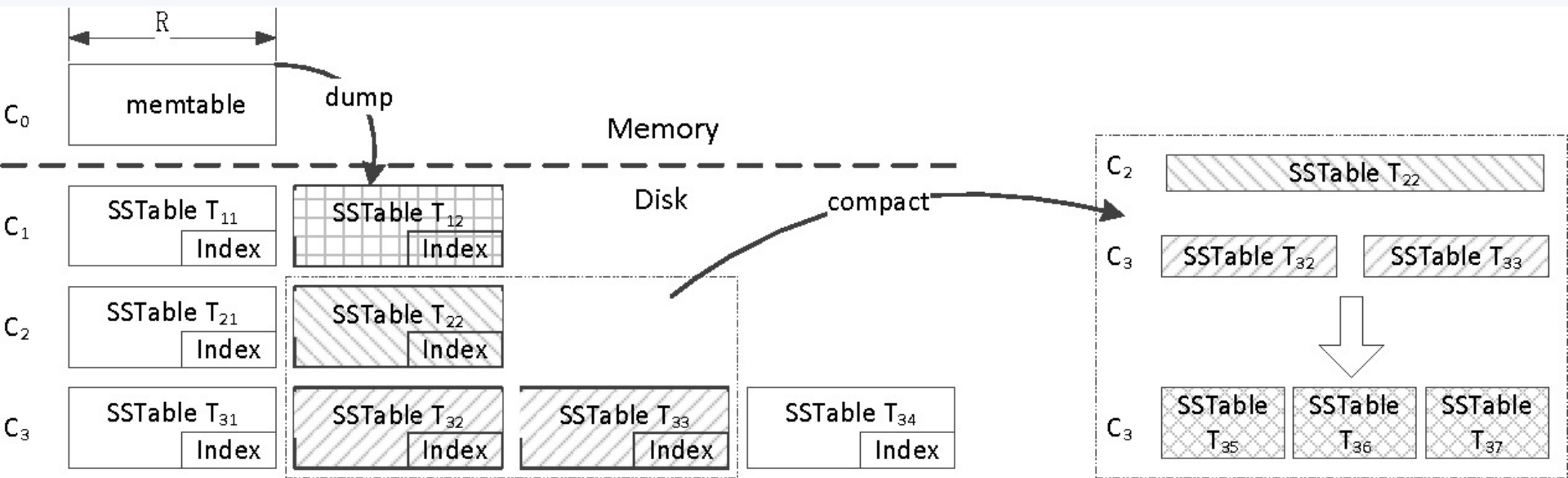
- » Writes are initially done to an in-memory structure called memtable, where the keys are kept sorted (random access of RAM is not expensive). Once the table “fills up”, it’s persisted in disk as an immutable (read-only) file





Log Structured Merge Tree

- » If tables are full, a compaction algorithm is invoked.



(a) LSM-tree data structure



(b) SSTable layout





Log Structured Merge Tree

- » Layer by Layer Compaction
- » Each level is k times larger than the previous one. In LevelDB [4], level L has a (10^L) MB size limit (that is, 10MB for level 1, 100MB for level 2, etc).
- »

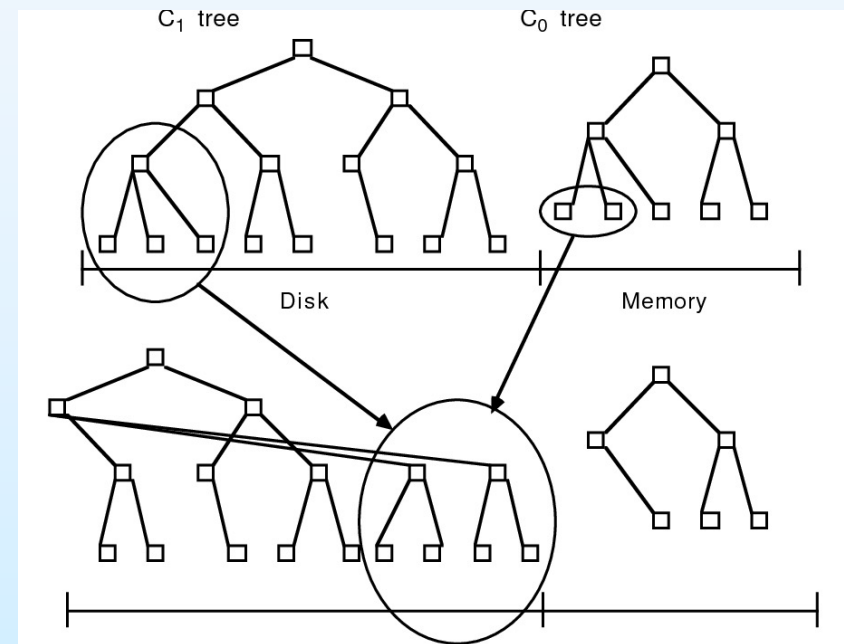
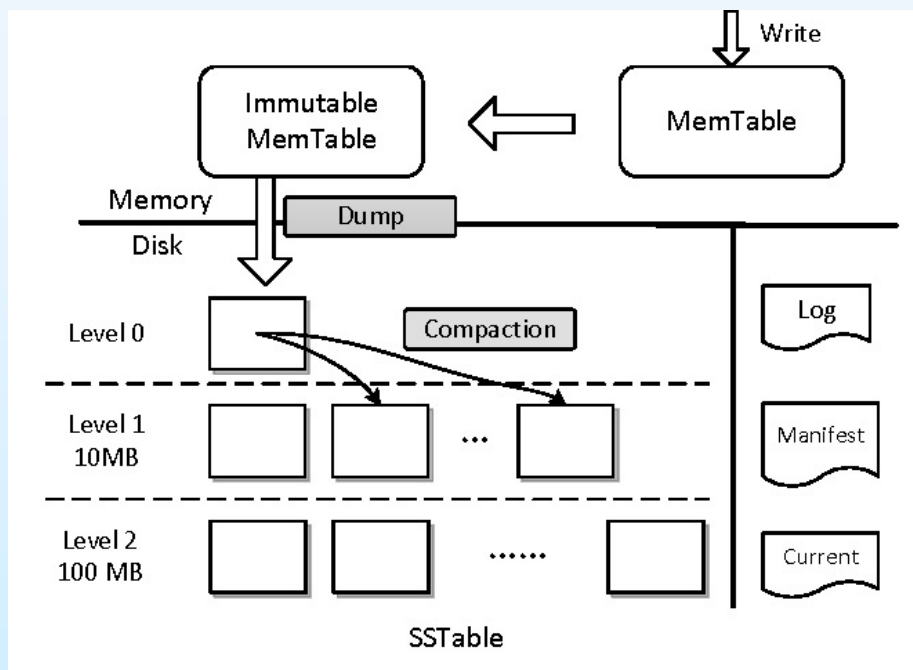


Figure 2.2. Conceptual picture of rolling merge steps, with result written to disk.



The Sun Network File System (NFS)

- An implementation and a specification of a software system for accessing remote files across LANs (or WANs)
- The implementation is part of the Solaris and SunOS operating systems running on Sun workstations using an unreliable datagram protocol (UDP/IP protocol and Ethernet)





NFS (Cont.)

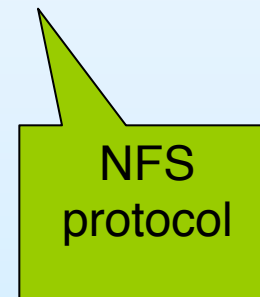
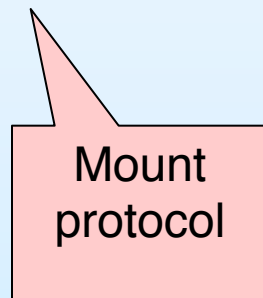
- Interconnected workstations viewed as a set of **independent machines** with **independent file systems**, which allows sharing among these file systems in a transparent manner
 - A remote directory is mounted over a local file system directory
 - ▶ The mounted directory looks like an **integral subtree** of the local file system, replacing the subtree descending from the local directory
 - Specification of the remote directory for the mount operation is nontransparent; the host name of the remote directory has to be provided
 - ▶ Files in the remote directory can then be accessed in a **transparent** manner
 - Subject to access-rights accreditation, potentially any file system (or directory within a file system), can be mounted remotely on top of any local directory





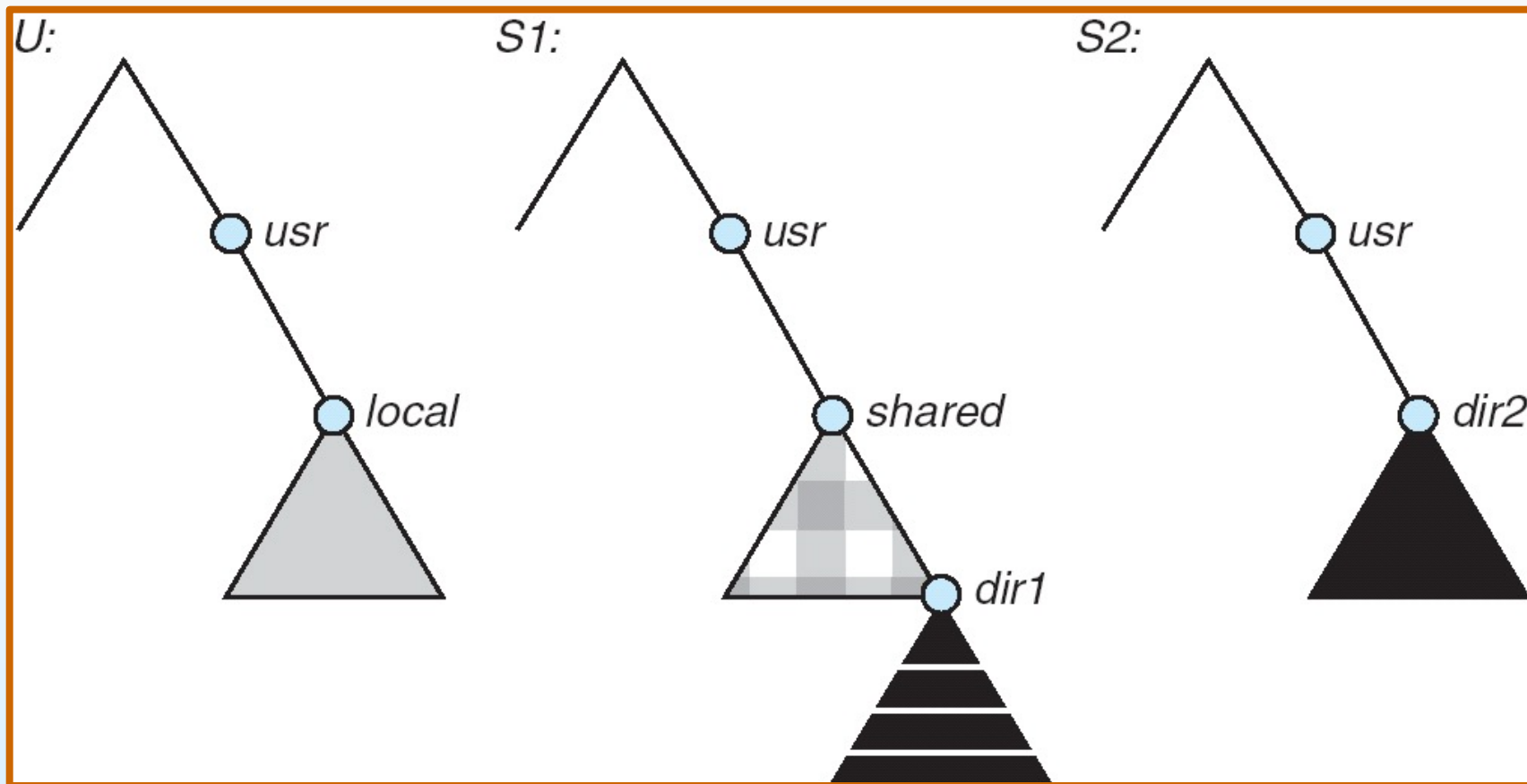
NFS (Cont.)

- NFS is designed to operate in a **heterogeneous** environment of different machines, operating systems, and network architectures; the NFS specifications independent of these media
- This independence is achieved through the use of RPC primitives built on top of an External Data Representation (XDR) protocol used between two implementation-independent interfaces
- The NFS specification distinguishes between the services provided by a mount mechanism and the actual remote-file-access services



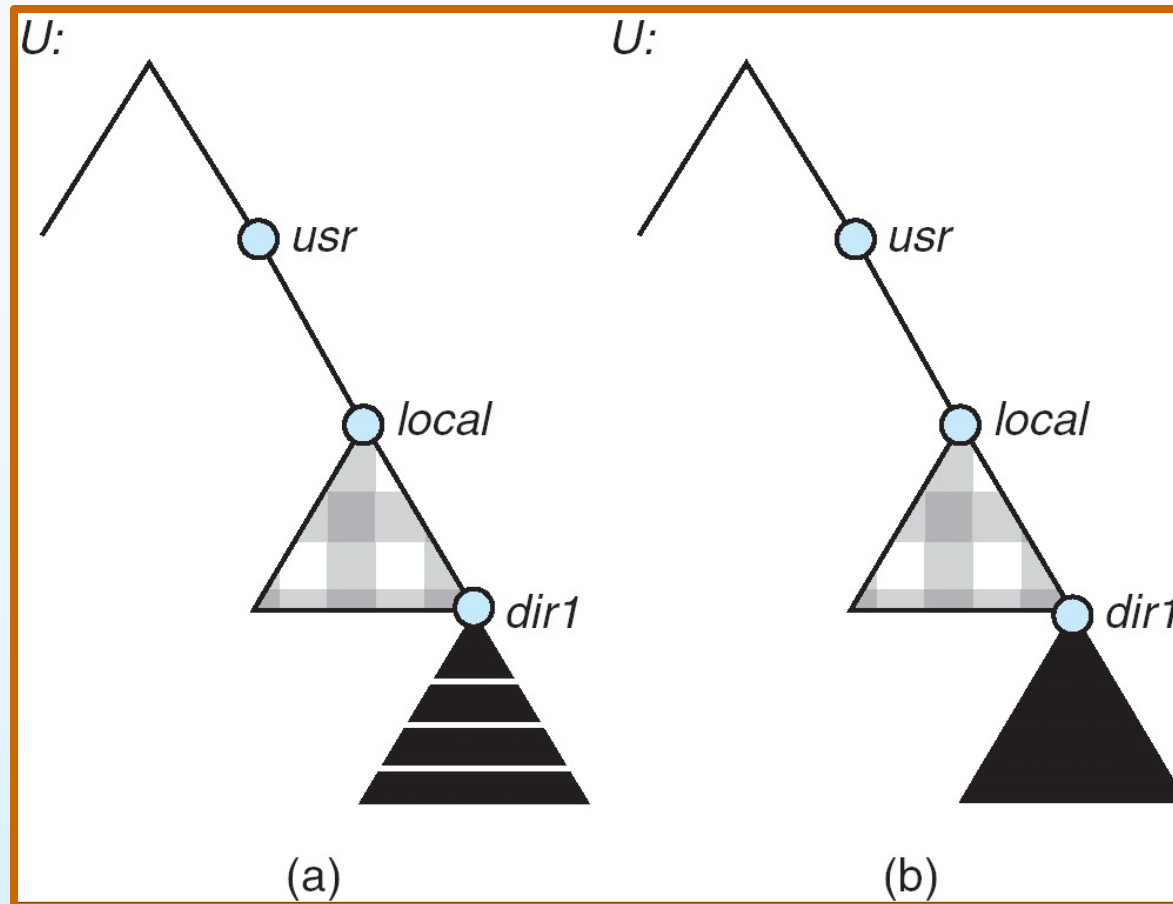


Three Independent File Systems





Mounting in NFS



Mounts

Mount S1:/usr/shared
Over U:/usr/local

Cascading mounts are allowed

Then mount S2:/usr/dir2
Over U:/usr/local/dir1





NFS Mount Protocol

- Establishes initial logical connection between server and client
- Mount operation includes name of remote directory to be mounted and name of server machine storing it
 - **Mount request** is mapped to corresponding RPC and forwarded to mount server running on server machine
 - **Export list** – specifies local file systems that server exports for mounting, along with names of machines that are permitted to mount them
- Following a mount request that conforms to its export list, the server returns a **file handle**—a key for further accesses
- File handle – a file-system identifier, and an inode number to identify the mounted directory within the exported file system
- The **mount** operation changes only the user's view and does not affect the server side





NFS Protocol

- ❑ Provides a set of remote procedure calls for remote file operations. The procedures support the following operations:
 - ❑ searching for a file within a directory
 - ❑ reading a set of directory entries
 - ❑ manipulating links and directories
 - ❑ accessing file attributes
 - ❑ reading and writing files
- ❑ NFS servers are **stateless**; each request has to provide a full set of arguments (unique file id, absolute offset inside file)
(NFS V4 is just coming available – very different, stateful)
- ❑ Modified data must be committed to the server's disk before results are returned to the client (lose advantages of caching)
- ❑ The NFS protocol does not provide concurrency-control mechanisms





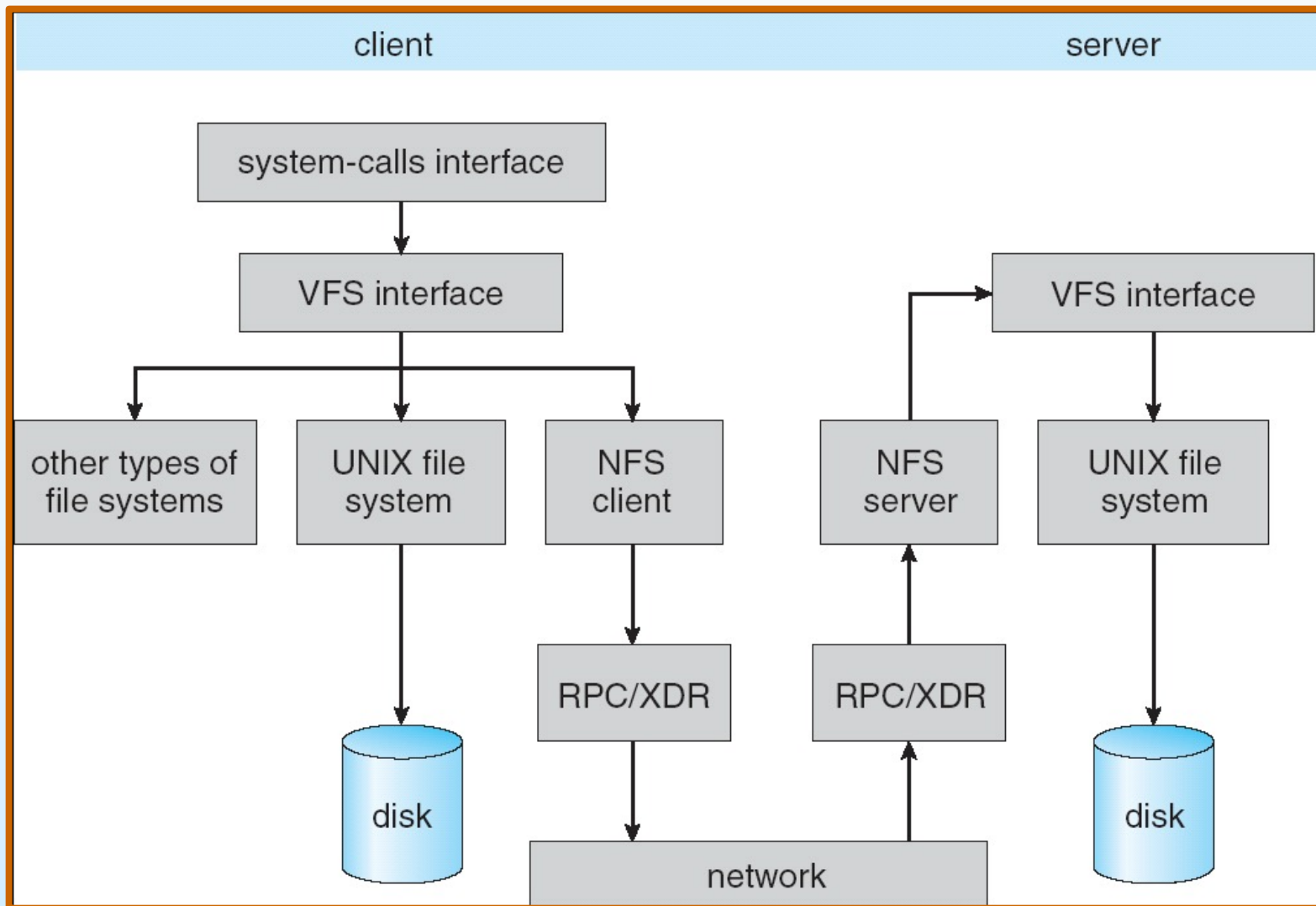
Three Major Layers of NFS Architecture

- UNIX file-system interface (based on the **open**, **read**, **write**, and **close** calls, and **file descriptors**)
- *Virtual File System* (VFS) layer – distinguishes local files from remote ones, and local files are further distinguished according to their file-system types
 - The VFS activates file-system-specific operations to handle local requests according to their file-system types
 - Calls the NFS protocol procedures for remote requests
- NFS service layer – bottom layer of the architecture
 - Implements the NFS protocol





Schematic View of NFS Architecture





NFS Path-Name Translation

- Performed by breaking the path into component names and performing a separate NFS lookup call for every pair of component name and directory vnode
- To make lookup faster, a directory name lookup cache on the client's side holds the vnodes for remote directory names





NFS Remote Operations

- Nearly one-to-one correspondence between regular UNIX system calls and the NFS protocol RPCs (except opening and closing files)
- NFS adheres to the remote-service paradigm, but employs buffering and caching techniques for the sake of performance
- **File-blocks cache** – when a file is opened, the kernel checks with the remote server whether to fetch or revalidate the cached attributes
 - Cached file blocks are used only if the corresponding cached attributes are up to date
- **File-attribute cache** – the attribute cache is updated whenever new attributes arrive from the server
- Clients do not free delayed-write blocks until the server confirms that the data have been written to disk





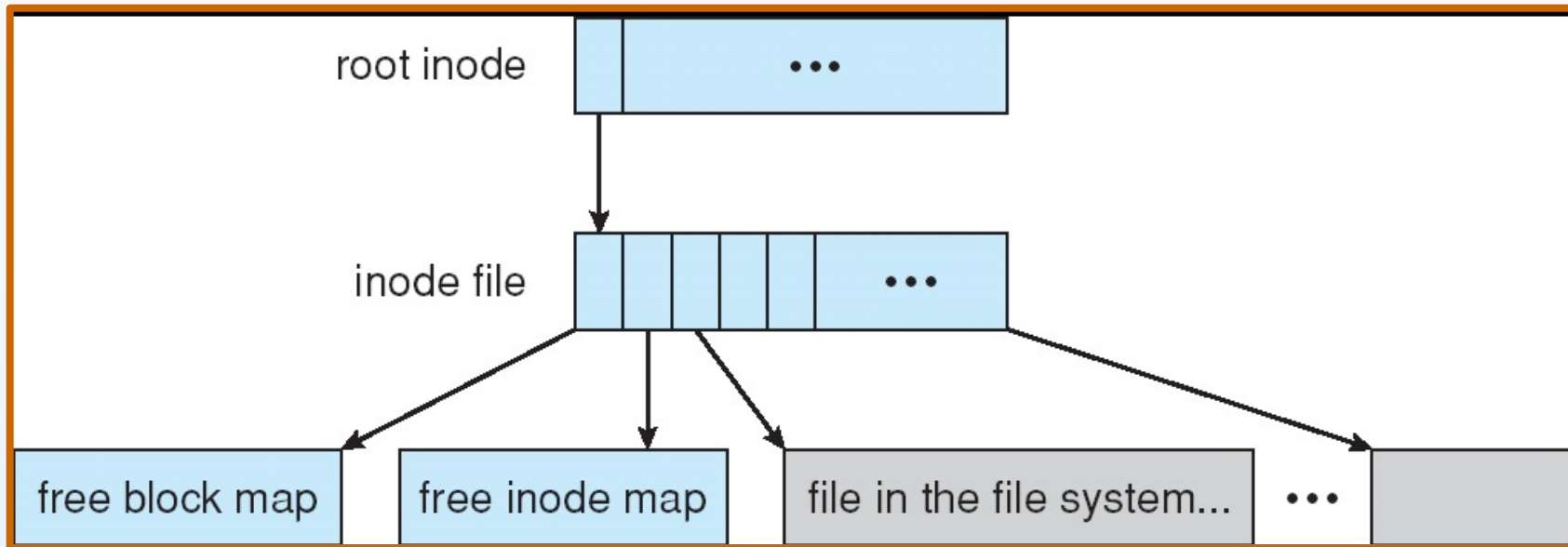
Example: WAFL File System

- Used on Network Appliance “Filers” – distributed file system appliances
- “Write-anywhere file layout”
- Serves up NFS, CIFS, http, ftp
- Random I/O optimized, write optimized
 - NVRAM (flash memory) for write caching
- Similar to Berkeley Fast File System, with extensive modifications



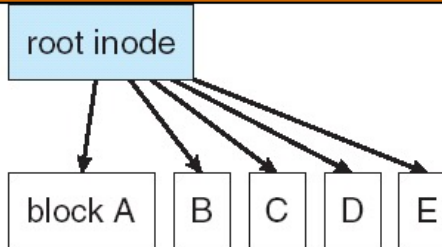


The WAFL File Layout

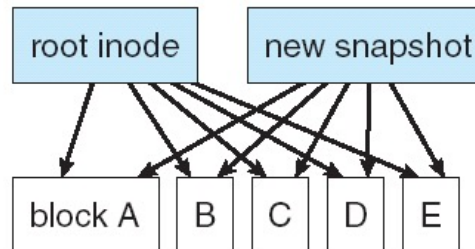




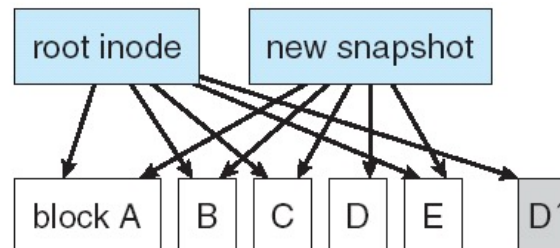
Snapshots in WAFL



(a) Before a snapshot.



(b) After a snapshot, before any blocks change.



(c) After block D has changed to D'.



End of Chapter 11





Homework

- » *Consider a system with a 32-bit logical address space. If the page size in such a system is 4 KB (2^{12}), then a page table may consist of up to 1 million entries ($2^{32}/2^{12}$). Assuming that each entry consists of 4 bytes, what is the page table size?*
- » In a 32-bit system with memory page size of 4KB ($2^2 * 2^{10}$ B), the maximum number of pages a process could have will be $2^{(32-12)} = 1\text{M}$. Each process *thinks* it has access to all physical memory. In order to translate all 1M virtual memory addresses to physical memory addresses, a process may need to store 1 M PTEs, that is 4MB.





Homework

- » In a 64 bit machine, with 256 MB RAM, and a 4KB page size, how many entries will there be in the page table if its inverted?

- » $1\text{MB} = 2^{20}$
- » $256\text{MB} = 2^{28}$
- » $4\text{KB} = 2^{12}$
- » $2^{28}/2^{12} = 2^{16}$





Homework

- » If the total number of available frames is 50, and there are 2 processes one of 10 pages and the other of 5 pages then how much of memory would be proportionally allocated to each of these processes?
- » 2:1
- » 33 and 16 frames respectively





Homework

- » If the hit ratio to a TLB is 80%, and it takes 15 nanoseconds to search the TLB, and 150 nanoseconds to access the main memory, then what must be the effective memory access time in nanoseconds?

- » $0.8 \cdot (15 + 150) + 0.2 \cdot (15 + 300) = 195$





Homework

- » Assume a page reference string for a process with m frames (initially all empty). The page reference string has length p with n distinct page numbers occurring in it. For any page-replacement algorithms, what is a lower bound & an upper bound on the number of page faults?
- » n, p

